

GROUP TECH BRIEF / TIBO TALK SERIES #02

Agent 的“身体”是什么

Tibo + Ed Bayes x Software Engineering Daily

播客 / 完整逐字稿 • 52 分钟 • 会谈日期 2026.01.29

一句话结论

这场把 Codex 的系统边界讲得最清楚：模型像“脑”，Harness 像“身体”；真正可用的 Coding Agent 是 Model + Harness + Sandbox + 产品交互共同进化的结果，而安全并不是外挂，而是执行能力的一部分。

四个最值得关注的信号

01 Co-evolution 模型、Harness、产品界面三者一起迭代，不存在稳定不变的外壳。	02 Sandbox-first 网络和文件系统默认受限，让 Agent 能大胆试错又不把副作用放大。
03 权限 UX 安全不是简单“禁掉”，而是细粒度权限、read-only / agent mode 与用户控制。	04 Continuous agent 当 Agent 可以长期不间断运行，现有回合式 UI 本身会成为瓶颈。

说明：下文将“访谈观点”“技术解读”“方向推演”分开呈现；预测不视为正式产品承诺。

OVERVIEW

访谈背景与章节地图

受访者：Thibault “Tibo” Sottiaux，OpenAI Core Product / Codex 相关负责人。

对谈方：Kevin Ball；嘉宾 Ed Bayes（Codex Product Designer）

形式：播客 / 完整逐字稿；**时长：**52 分钟

- 访谈观点
- 技术解读
- 方向推演
- 外部核验

章节地图 / 讨论顺序

时间 / 顺序	主题
00:02	Tibo / Ed 的背景与 Codex 起点
00:04	Model 与 Harness co-evolution
00:05	“brain + body” 比喻
00:06	为什么 Agent 默认进 sandbox
00:07	安全与用户体验之间的张力
00:08	从 Cloud VM 到 CLI / VS Code 仍坚持 safe-by-default
后续	多 Agent、开源 CLI、模型专用化、延迟与性能

阅读方法

先看 Tibo 明确表达的系统设计原则，再看这些原则对 Agent Runtime、Serving、Sandbox、Memory、Multi-agent 或软件工程流程的含义。不要把方向判断直接当作已经落地的功能。

CORE IDEAS

1. Agent 的能力边界是系统边界

- 如果只拿一个模型 API 去比较 Codex，很容易低估 Harness 的影响：工具描述、环境反馈、权限、文件系统、终端、上下文组织都会改变模型能否把“会想”变成“做成”。
- Tibo 用“brain/body”形容 Model/Harness：身体不仅负责执行动作，还要保证动作在可控边界内发生。于是 Sandbox 不再只是安全团队的要求，而是允许 Agent 探索环境的前提。
- 这也解释了为什么同一模型放进不同 Agent 产品，端到端成功率可以差很多：行为闭环本身就是能力。

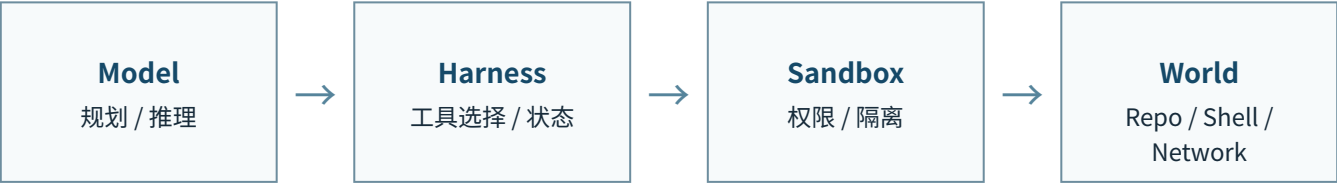
核心判断：这场把 Codex 的系统边界讲得最清楚：模型像“脑”，Harness 像“身体”；真正可用的 Coding Agent 是 Model + Harness + Sandbox + 产品交互共同进化的结果，而安全并不是外挂，而是执行能力的一部分。

2. Safe-by-default 不是牺牲能力，而是扩大可授权范围

- Codex 的思路是默认限制网络、文件系统和命令权限；用户可以逐步授予更细粒度的能力。这样即便模型犯错，blast radius 也被限制。
- 从最早的隔离 Cloud VM 到后来本地 CLI / VS Code，执行位置变了，但“先受限、再放权”的原则没有变。
- 对于长期 Agent，这个设计尤其重要：任务时间越长、工具调用越多，单次小概率错误的累计风险越高。

SYSTEM VIEW

把访谈观点翻译成系统结构



这里最容易误解的一点

Agent 的“能力”并不等于裸模型能力。对真实任务而言，更接近：

$$\text{Agent Utility} = \text{Model} \times \text{Context} \times \text{Tools} \times \text{Runtime} \times \text{Safety} \times \text{Verification}$$

任何一项接近 0，端到端体验都会坍塌；这也是为什么模型 benchmark 提升并不总能等比例转化为产品收益。

工程上应该关注的闭环

Observe	任务 trace、tool call、权限、失败位置、人工介入、最终结果。
Diagnose	区分是模型、Harness、环境、数据、延迟还是安全策略导致失败。
Improve	把问题放回最合适的层修，而不是默认继续堆 prompt 或规则。
Verify	用可重复 eval 证明端到端成功率、成本和 latency 真正改善。

AI INFRA TAKEAWAYS

对 AI Infra / 安全执行层的启发

Blast radius	Agent 安全指标不只看“危险行为概率”，还要乘以一次错误能影响的资源范围。Sandbox 是直接压缩后者。
权限缓存	用户批准过的命令如何被精细记录和复用，会影响每次任务的交互延迟与安全性。
本地 vs 云端	本地有低摩擦上下文与环境；云端更易并行、隔离、恢复。未来很可能是混合执行。
尾延迟	频繁确认会放大 human-in-the-loop latency。高风险动作再确认、低风险动作自动放行需要风险分层。
持续 Agent	24/7 运行需要 watchdog、budget、checkpoint、credential lifecycle、审计日志和可撤销动作。

给组内讨论的一个统一框架

把每项 Agent 优化问成四个问题：①端到端成功率提升多少；②wall-clock 缩短多少；③总算力/Token/工具成本增加多少；④风险与人工验收成本是否变高。单看模型分数、Kernel 峰值或单 Token 价格都不够。

$$T_{total} = T_{queue} + T_{prefill} + T_{decode} + T_{tool} + T_{network} + T_{environment} + T_{verification} + T_{human_wait}$$

WHAT TO BELIEVE / WHAT TO DO

需要打折看的部分

- “Sandbox 更安全”不等于零风险：凭证泄露、间接 prompt injection、被授权工具的业务副作用仍然存在。
- 这场访谈发生在 2026 年 1 月，产品形态后来继续演进，因此应把它当作设计原则来源，而不是今天 UI 的使用说明。
- 部分逐字稿为第三方转写，个别容器/实现名可能有误，不在本报告中作为结论。

给组员的三个可执行动作

1. 把 Agent 的 tool permission 划分 read / write / destructive / external-send 四级，并记录升级原因。
2. 端到端 profiling 里加入 approval wait time，避免只优化模型 latency。
3. 为长期任务定义“可撤销性”：哪些写操作必须有 snapshot / transaction / dry-run。

本场会谈的最终一句话

这场把 Codex 的系统边界讲得最清楚：模型像“脑”，Harness 像“身体”；真正可用的 Coding Agent 是 Model + Harness + Sandbox + 产品交互共同进化的结果，而安全并不是外挂，而是执行能力的一部分。

原始资料

<https://softwareengineeringdaily.com/podcasts/openai-and-codex-with-thibault-sottiaux-and-ed-bayes/>

<https://podscan.fm/podcasts/software-engineering-daily/episodes/openai-and-codex-with-thibault-sottiaux-and-ed-bayes>

整理日期：2026-09-10。报告为技术阅读笔记，不代表 OpenAI 官方路线承诺；产品状态以最新官方文档为准。